Rapport Synthétique : AIESEC Hub

Plateforme collaborative de gestion d'événements et de retours associatifs

Projet de Développement Web Avancé (Stack MERN) – Année universitaire 2025 / 2026

Étudiant(e) :Abir Rouag | Groupe / Filière : 5 DS

Application en ligne :https://aiesec-events-2bxwtj5d1-abir-rouag.vercel.app/ |

API : https://aiesec-events.onrender.com/ |

Dépôt GitHub : https://github.com/abirrouag/AIESEC-EVENTS

---

1. Présentation du projet

Au sein d'organisations étudiantes internationales comme **AIESEC**, la coordination des comités locaux repose sur une communication rapide autour de projets phares (villages globaux, conférences de leadership, programmes d'échanges). Les informations sur les événements sont toutefois dispersées et les retours qualitatifs des membres sont rarement structurés.

AIESEC Hub répond à ce besoin : c'est une application web monopage (SPA), épurée et sécurisée, qui permet aux membres du comité d'annoncer leurs activités et de recueillir les avis de leur communauté.

Fonctionnalités

Mode Visiteur (non connecté)

- Consultation en lecture seule de la liste chronologique des événements.
- Lecture de tous les feedbacks associés à chaque événement.
- Accès aux pages d'inscription et de connexion ; aucune publication ni suppression possible.

Mode Membre (authentifié via JWT)

- Création d'événements (titre, date, description) via un formulaire contrôlé.
- Suppression de **ses propres** événements uniquement.
- Publication de commentaires sur n'importe quel événement, et suppression de **ses propres** feedbacks uniquement.
- Session persistante (`localStorage`) avec reconnexion automatique.

---

2. Architecture et choix technologiques

Le projet suit l'architecture **MERN** en trois couches :

Couche	Technologie	Rôle
Client	React 18 + Vite + Axios	Interface SPA, intercepteur JWT, session `localStorage`
Serveur	Node.js + Express	API REST JSON, middlewares CORS / `express.json` / `protect`
Données	MongoDB Atlas (M0) + Mongoose	Collections Users, Events, Feedbacks

Justification : Node/Express pour des API REST légères et asynchrones ; MongoDB/Mongoose pour la flexibilité du schéma, la validation et les jointures via `.populate()` ; React + Vite pour un démarrage quasi instantané grâce aux modules ES natifs ; Axios pour centraliser la configuration HTTP et injecter automatiquement le jeton Bearer.

Les échanges client-serveur se font en HTTPS via une API REST (routes `/api/auth`, `/api/events`, `/api/feedbacks`).

---

3. Modélisation des données : schémas Mongoose

Trois entités reliées par des relations **One-to-Many (1 → N)** :

- un **User** possède plusieurs **Events** ;
- un **User** possède plusieurs **Feedbacks** ;
- un **Event** reçoit plusieurs **Feedbacks**.

3.1. Modèle `User` (`server/models/User.js`)

```
const userSchema = new mongoose.Schema({
  name:     { type: String, required: true, trim: true },
  email:    { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true } // haché via bcrypt
}, { timestamps: true });
```

3.2. Modèle `Event` (`server/models/Event.js`)

```
const eventSchema = new mongoose.Schema({
  title:       { type: String, required: true, trim: true },
  description: { type: String, required: true },
  date:        { type: Date, required: true },
  author:      { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });
```

3.3. Modèle `Feedback` (`server/models/Feedback.js`)

```
const feedbackSchema = new mongoose.Schema({
  text:      { type: String, required: true, trim: true },
  event:     { type: mongoose.Schema.Types.ObjectId, ref: 'Event', required: true },
  author:    { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now }
});
```

3.4. Jointure dynamique avec `.populate()`

Pour afficher le nom de l'auteur sans exposer son mot de passe :

```
const events = await Event.find().populate('author', 'name').sort({ date: 1 });
```

---

4. Sécurité et API back-end

- **Mots de passe :** salage (`genSalt(10)`) puis hachage irréversible avec `bcryptjs` ; vérification à la connexion par `bcrypt.compare()`.
- **JWT :** jeton signé contenant `{ id: user._id }`, clé secrète `JWT_SECRET`, validité de 7 jours.
- **Middleware `protect` :** extrait l'en-tête `Authorization: Bearer <token>`, renvoie `401` si le jeton est absent, vérifie la signature (`jwt.verify`), charge l'utilisateur (`select('-password')`), l'injecte dans `req.user` puis appelle `next()`.
- **Autorisation par propriété :** seul l'auteur d'un événement ou d'un feedback peut le modifier ou le supprimer.

Matrice des routes

Méthode	Route	Rôle	Accès	Succès
POST	`/api/auth/register`	Créer un compte	Public	201
POST	`/api/auth/login`	Connexion + émission du JWT	Public	200
GET	`/api/events`	Liste des événements	Public	200
POST	`/api/events`	Publier un événement	Protégé (JWT)	201
PUT	`/api/events/:id`	Modifier son événement	Protégé (auteur)	200
DELETE	`/api/events/:id`	Supprimer son événement	Protégé (auteur)	200
GET	`/api/feedbacks/:eventId`	Avis d'un événement	Public	200
POST	`/api/feedbacks`	Déposer un avis	Protégé (JWT)	201
DELETE	`/api/feedbacks/:id`	Supprimer son avis	Protégé (auteur)	200

---

5. Développement front-end

Organisation : `components/` (Navbar, EventCard, EventForm, FeedbackList), `pages/` (Home, Login, Register), `api.js` (instance Axios + intercepteur), `App.jsx` (état racine et routage), `index.css` (charte AIESEC, couleur principale #037EF3).

Intercepteur Axios : ajoute automatiquement le jeton du `localStorage` à chaque requête :

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
```

```
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

État et rendu conditionnel : `useState` gère les champs, la liste d'événements et l'utilisateur actif ; `useEffect` charge les données au montage. Les boutons de suppression et les formulaires ne s'affichent que si l'identifiant de l'utilisateur connecté correspond à `event.author._id` ou `feedback.author._id`.

---

6. Déploiement cloud

Service	Plateforme	Configuration clé
Base de données	MongoDB Atlas (M0)	Utilisateur applicatif, accès réseau, URI `mongodb+srv://...`
Back-end	Render (Web Service)	Racine `server`, build `npm install`, start `node server.js`, variables `PORT`, `MONGO_URI`, `JWT_SECRET`, `CLIENT_URL`
Front-end	Vercel	Racine `client`, preset Vite, variable `VITE_API_URL`, `vercel.json` redirigeant vers `/index.html`

---

7. Validation et tests

La suite de tests (`server/test-server.js`) a validé :

1. `GET /` renvoie `200 OK` (route de santé).
2. `POST /api/auth/register` sans données renvoie `400 Bad Request`.
3. `POST /api/events` et `POST /api/feedbacks` sans jeton renvoient `401 Unauthorized`.
4. Hachage/vérification `bcrypt` et encodage/décodage `jwt` conformes.
5. Build Vite de production sans erreur ni avertissement.

---

8. Captures d'écran de l'application en ligne

Insérer chaque capture dans l'espace prévu sous sa légende (largeur conseillée : pleine page, environ 15 cm).

Figure 1 : Page d'accueil – vue visiteur

Bannière AIESEC Hub, liste chronologique des événements et invitation à se connecter.

> *[Insérer ici la capture d'écran de la page Home en mode visiteur]*

Figure 2 : Formulaires de connexion et d'inscription

Validation des champs et messages d'erreur contextuels.

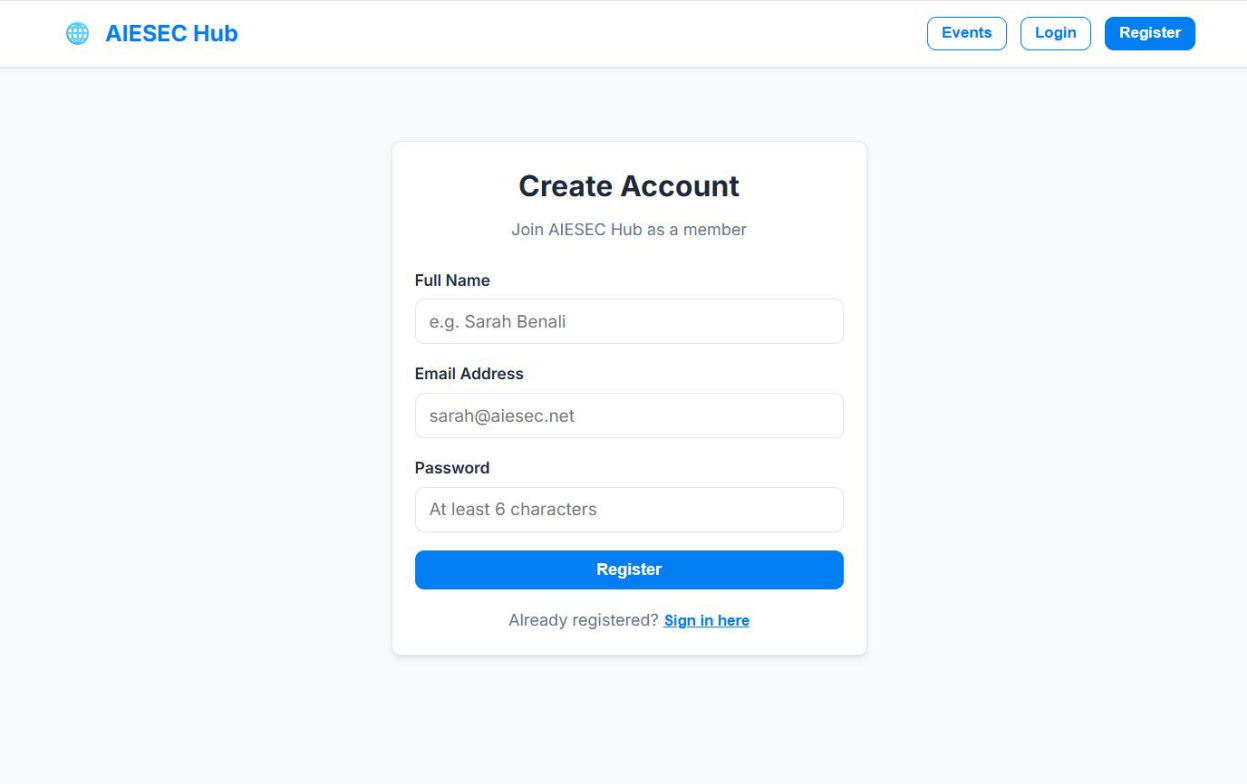


Figure 3 : Tableau de bord membre et ajout d'événement

Message « Hi, [Nom] », formulaire d'ajout d'événement et bouton Déconnexion.

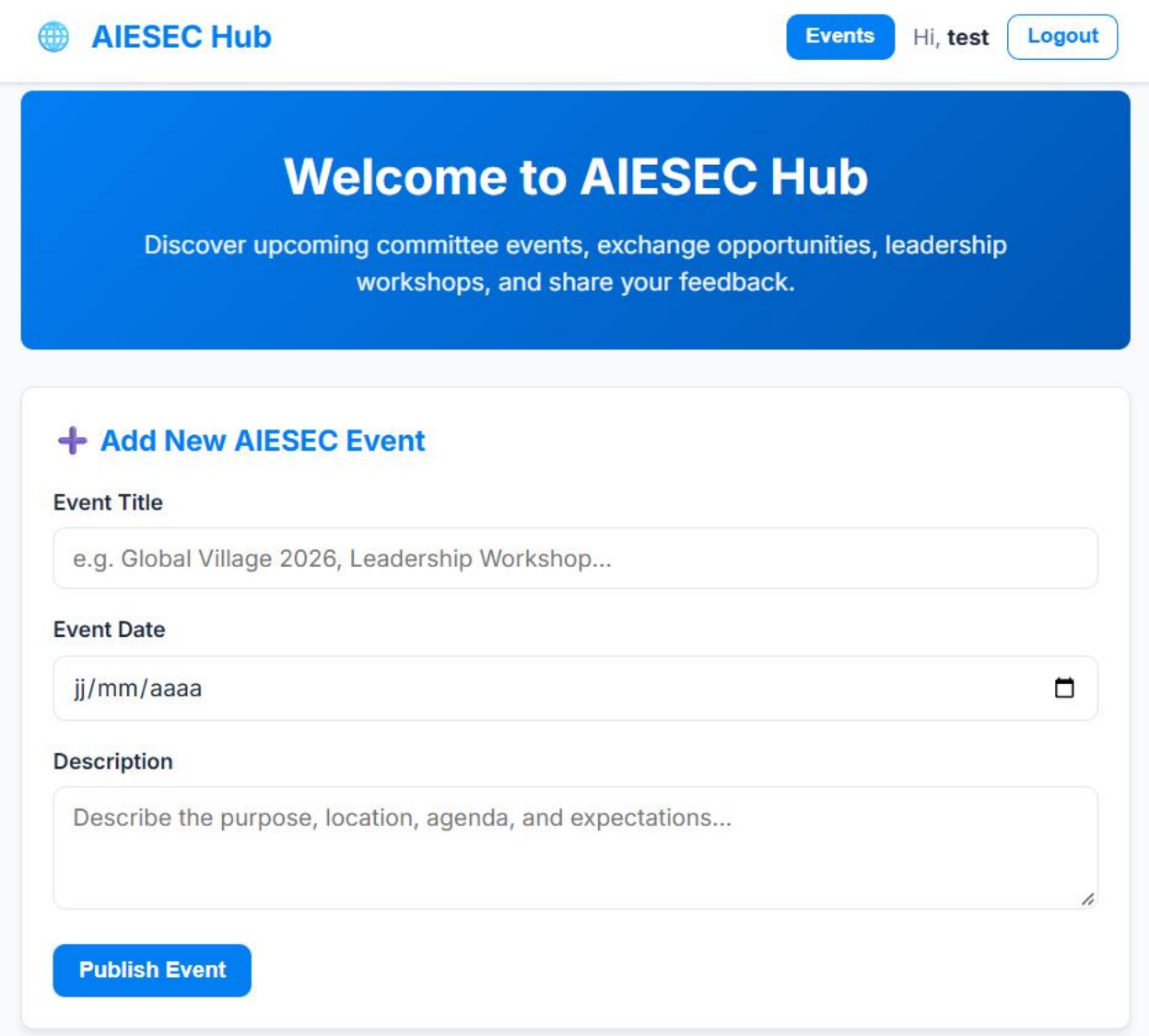


Figure 4 : Consultation et ajout de feedbacks

Commentaires avec auteur, date et bouton de suppression sur ses propres avis.

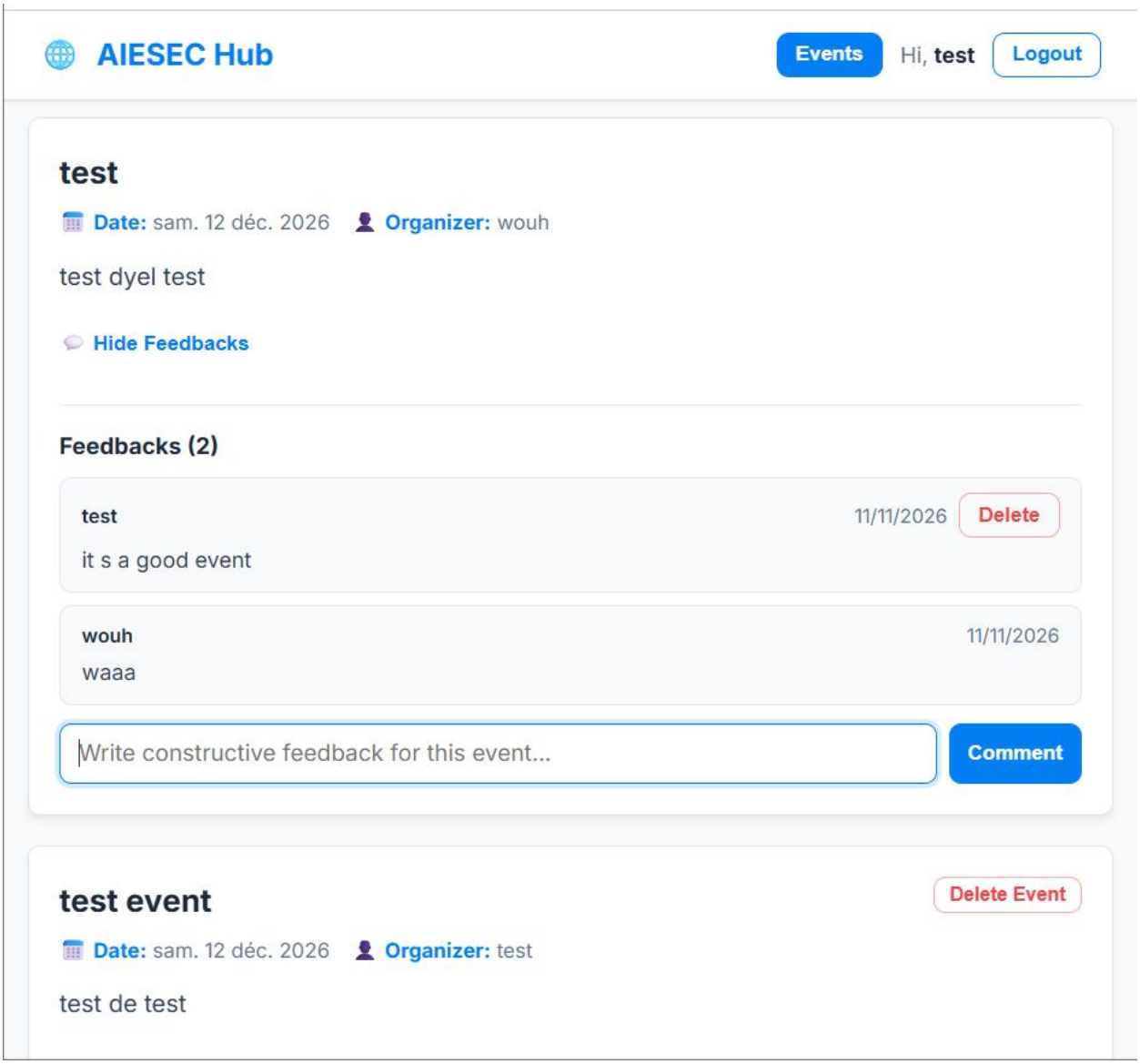
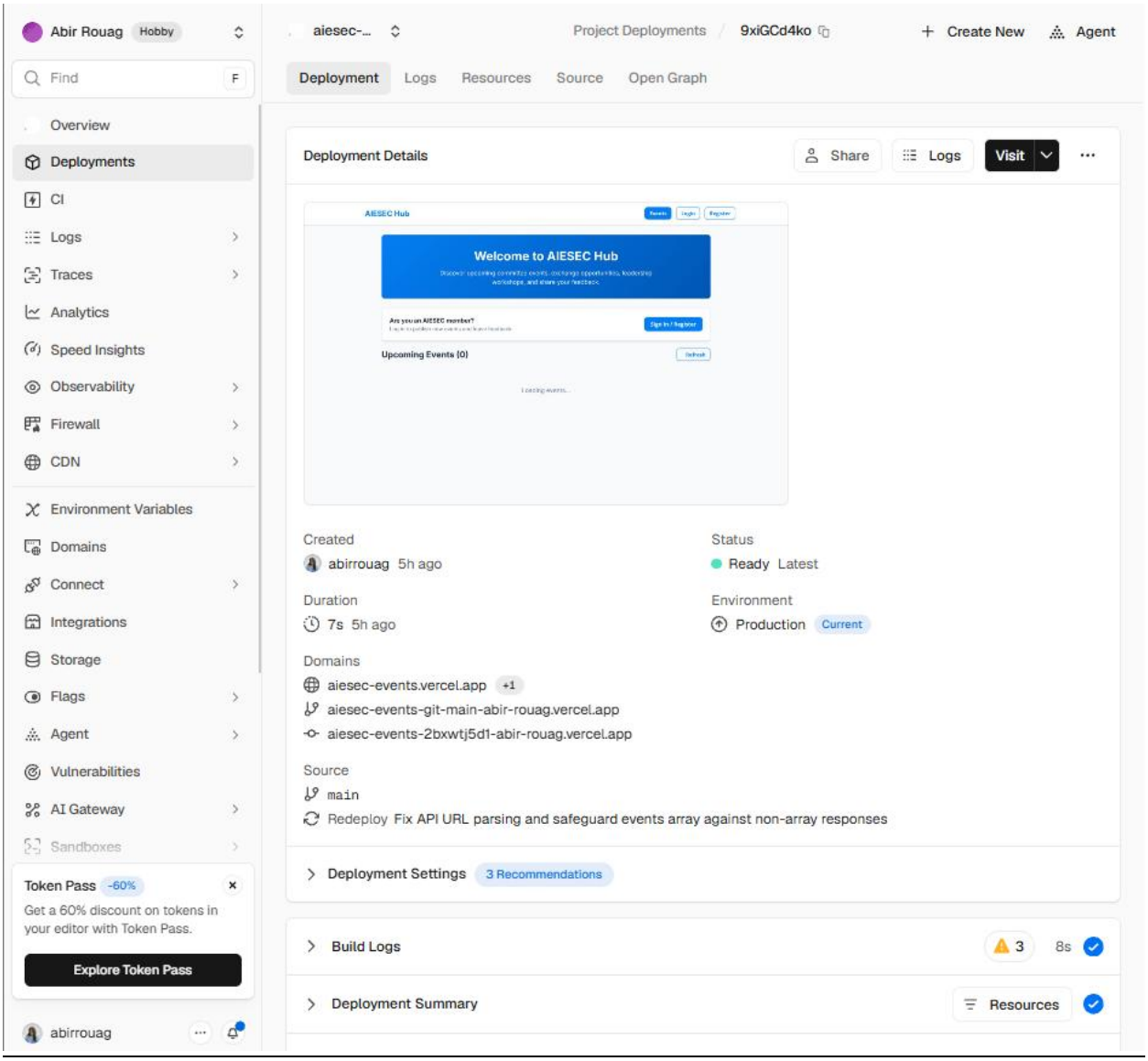
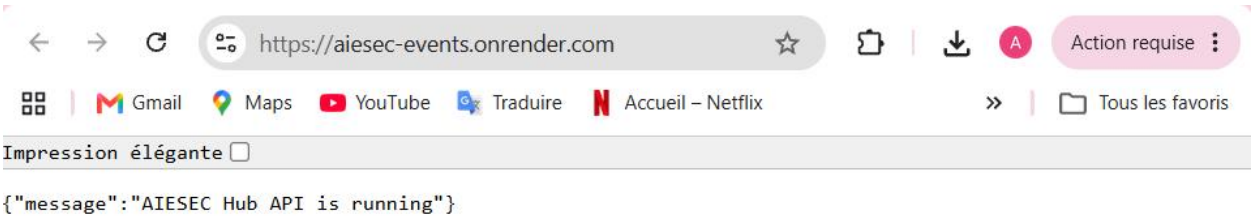


Figure 5 : Tableaux de bord de déploiement (Render et Vercel)

Statut « Live » du service API sur Render et déploiement « Ready » sur Vercel.


Abir Rouag
Hobby
Find
F
Overview
Deployments
CI
Logs
Traces
Analytics
Speed Insights
Observability
Firewall
CDN
Environment Variables
Domains
Connect
Integrations
Storage
Flags
Agent
Vulnerabilities
AI Gateway
Sandboxes
Token Pass
-60%
Get a 60% discount on tokens in your editor with Token Pass.
Explore Token Pass
abirrouag
aiesec-...
Project Deployments
9xiGCd4ko
Create New
Agent
Deployment
Logs
Resources
Source
Open Graph
Deployment Details
Share
Logs
Visit
AIESEC Hub
Welcome to AIESEC Hub
Upcoming Events (0)
Created
abirrouag 5h ago
Status
Ready Latest
Duration
7s 5h ago
Environment
Production
Current
Domains
aiesec-events.vercel.app
+1
aiesec-events-git-main-abir-rouag.vercel.app
aiesec-events-2bxwtj5d1-abir-rouag.vercel.app
Source
main
Redeploy Fix API URL parsing and safeguard events array against non-array responses
Deployment Settings
3 Recommendations
Build Logs
3
8s
Deployment Summary
Resources



---

9. Conclusion et perspectives

AIESEC Hub valide les objectifs pédagogiques et techniques du projet : maîtrise de la pile MERN, bonnes pratiques de sécurité (hachage, JWT, variables d'environnement), séparation nette des droits entre visiteurs et membres, et architecture déployée en continu sur le cloud en HTTPS.

Perspectives :

1. Catégories d'événements (O-Xchange, Youth Speak, Team Building).
2. Système de notation par étoiles (1 à 5) en complément du commentaire.
3. Téléversement d'affiches d'événements sur un stockage cloud (ex. Cloudinary).